

FixProof: A Validation-Oriented System for AI-Generated Vulnerability Remediation

Tony Tran

Progress Report 3 - September 28, 2026

Problem Statement:

AI coding tools can produce insecure code, while a proposed security repair can leave a weakness exploitable or change intended application behavior. Disappearance of a static-analysis warning alone is incomplete evidence of a correct repair. Pearce et al. (2022) found approximately 40% of 1,689 Copilot-generated programs vulnerable within their selected security scenarios. Existing work also studies LLM vulnerability repair and validation feedback (Pearce et al., 2023; Kulsum et al., 2024). FixProof asks how separately recorded static, runtime-security, functional, and human-review evidence can support a patch decision.

The evaluated scope is deliberately narrow: three purpose-built JavaScript/Express applications with reflected XSS (CWE-79), SQL injection (CWE-89), and path traversal (CWE-22). FixProof evaluates AI-generated repairs to these fixtures, not a representative corpus of AI-generated applications or production systems. This refinement addresses the original concern about concrete application construction, model role, and systematic evaluation.

The 15 repair attempts do not estimate how often AI-generated repairs succeed in production. Instead, this controlled benchmark evaluates whether separate evidence channels reveal information omitted by an initial pass/fail result. All 15 candidates passed the frozen primary security and functional checks, but the predefined and frozen supplemental evaluation later identified five behavioral-parity failures, ten robustness failures, and five inconclusive symlink-security observations. These results support the project goal at the workflow level: an AI-generated repair should be treated as an untrusted candidate and evaluated through preserved static, runtime, behavioral, policy, and human-review evidence. The measured percentages apply only to these three fixtures and are not projected to production repositories.

Solution Statement:

FixProof uses Python to orchestrate Semgrep CE scanning, finding normalization, focused remediation prompts, candidate application in a copied workspace, JavaScript syntax checks, SAST comparison, targeted security tests, and functional tests. Deterministic policy routes a candidate to REJECT, READY_FOR_HUMAN_REVIEW, or NEEDS_HUMAN_ADJUDICATION. The repair model proposes source changes but does not evaluate, approve, or deploy its own patch. Human conclusions are stored separately from automated decisions.

These choices are deliberate. Three minimal fixtures make vulnerability ground truth, attack behavior, and benign behavior reproducible within the practicum timeline, at the cost of production realism. A fixed model, prompt version, and five calls per CWE hold generation conditions constant while showing within-fixture variation; they do not provide independent application samples or model-to-model comparison. Semgrep supplies one preserved static signal, while targeted runtime and functional checks measure exploit behavior and intended behavior separately. Copied workspaces preserve study inputs but are not security sandboxes. Deterministic policy makes evidence handling repeatable, and a separate human boundary prevents the repair model from approving its own output.

Deliverables include the prototype, three versioned CWE benchmarks, preserved primary and supplemental evidence, a dashboard and CLI demonstration, reproducibility documentation, and the final presentation and report. The dashboard displays saved evidence; a controlled live replay is labeled separately from the frozen 15-attempt study.

Completed Tasks (Since Progress Report 2):

- Confirmed the historical primary-review checkpoint: all ten original SAST/runtime disagreement reviews are complete. Seven accepted the candidate under the original evidence; three requested additional testing. These reviews were already described in Video II and are background, not work newly completed after the September 13 Report 2 submission.
- Predefined and froze supplemental-v1 before candidate execution. It registered 28 cases across three CWEs and separated security, behavioral-parity, and new robustness oracles. Three vulnerable baselines were characterized before the same applicable cases were applied to all 15 saved primary candidates in disposable copies. No new model calls were made.
- Recorded 140 supplemental candidate-case observations: security 55 pass, zero fail, five inconclusive; behavioral parity 40 pass, five fail; robustness 25 pass, ten fail. These different categories are not pooled into a security success rate.
- Recorded three requested September 15 follow-up rejections for XSS 04, XSS 05, and path-traversal 01. On September 21, I reviewed XSS 01 and 02 against the same frozen missing-input criterion and recorded separate FOLLOW_UP_REJECT_CANDIDATE qualifications. Their original ACCEPT_CANDIDATE records and primary metrics remain unchanged.
- Documented limits in the supplement: four XSS candidates changed omitted-name output; traversal 01 rejected a valid in-root filename; all five SQLi candidates missed a newly defined repeated-parameter rule; selected traversal candidates missed new robustness rules. Windows could not create the symlink fixture, so five traversal security observations remain inconclusive rather than passing.
- Recorded nine additional September 22 supplemental human results after reviewing the linked patches and registered outcomes. I accepted SQLi 01-05 for the bounded SQL-injection repair because parameterized queries passed every registered security and parity case and returned no broadened rows; SQL-R01 remains a disclosed robustness failure because repeated usernames returned HTTP 200 with an empty array instead of HTTP 400. I requested more testing for traversal 02-05 because symlink-escape protection remains inconclusive; traversal 03 also failed three malformed-input robustness cases and traversal 05 failed the repeated-parameter case. These records do not alter frozen primary evidence or registered outcomes.
- Improved verification portability after an old-checkout absolute path prevented supplemental verification here. The verifier checks the registered source in the active checkout with matching hashes. On September 22, 119 automated tests passed; primary evidence verified 15/15 attempts and 10/10 original reviews; supplemental evidence verified three baselines and 15 candidates; and all 14 supplemental human packet/result bindings verified: five rejections, five bounded acceptances, and four requests for more testing.
- Generated and inspected the 15-candidate evidence appendix. Its primary and supplemental matrices preserve the original automated states, registered failures and inconclusive cases, all 14 later human decisions, and complete recorded rationales. It is a backup reference for Video III rather than a replacement for the report narrative.
- Reviewed repository structure, documentation, test-code snippets and saved results, and supplied human rationales. Video II was posted, and I completed the required Video II peer-feedback activity by submitting four feedback responses. Peer feedback remains a separate course deliverable rather than a FixProof evaluation outcome.
- Built, privacy-checked, and browser-validated a sanitized static evidence reference at <https://fixproof.netlify.app/>. It presents all 15 primary and supplemental candidate rows, ten original reviews, 14 later records, rationales, patch excerpts, the workflow figure, and the PDF appendix. It performs no model calls, scans, application execution, approval, or deployment and does not change any experimental denominator or result.

Reporting checkpoint: September 14-28, 2026, following the September 13 Report 2 submission. I estimate 1.5 hours per day for fifteen days, approximately 22.5 hours, spent mainly reviewing the file structure and documentation, validating test and saved-evidence results, reviewing candidate patches and code snippets, supplying human-review rationales, preparing the evidence appendix and Video III materials, completing four peer-feedback responses, building the sanitized public evidence reference, and aligning the report with the professor's feedback. This is my estimate, not a number inferred from commits or automated test time. Earlier reports recorded 31.5 hours plus a separate 13.5-hour September 2-10 period. The documented subtotal through September 28 is approximately 67.5 hours; it excludes any September 11-13 work not already included in Report 2.

Tasks for the Next Project Report:

- Summarize applicable themes from the four submitted Video II feedback responses and incorporate them when they materially improve Video III or the final report. Retain the submission receipt separately from the FixProof evidence.
- Post Video III by October 6 and complete Peer Feedback Report III by October 11 at 11:59 p.m. Retain both submission receipts and incorporate applicable feedback into the next project report.
- Expand the related-work comparison across benchmark type, repair model, validation oracle, retries, success criteria, and limits. Draft final-paper methods and results with the primary, pilot/control, and supplemental evidence clearly separated.
- Prepare Video III around XSS 03 versus XSS 04. Use the read-only public evidence reference to show the patch, independent evidence channels, XSS-P01 parity result, and separate human boundary. Keep raw saved artifacts as technical backup, and do not call the website or a pilot replay a rerun of the frozen primary study.
- Decide whether to retain the symlink limitation or conduct a separately dated run on a capable environment. Before final delivery, extend the clean-archive checklist to verify supplemental evidence and all 14 human-record bindings as well as primary evidence.
- Create a clean versioned release snapshot before the final demonstration. Exclude local dependencies, caches, generated logs, and secrets; then rerun the primary, supplemental, human-record, and full automated checks from the packaged copy.

Questions I Have or Issues I Am Running Into:

The central issue is interpretation across evidence collected at different times. The original XSS 01-02 acceptances reflected the primary suite, but later XSS-P01 showed the same omitted-name parity failure as rejected XSS 04-05. I applied the frozen criterion consistently through separate later rejections; I did not rewrite the original decisions. A passing registered XSS attack suite does not establish full behavioral preservation.

The path-traversal symlink case could not execute on this Windows environment, leaving five security observations inconclusive. I therefore recorded requests for more testing rather than claiming symlink-safe traversal protection for attempts 02-05. Traversal 03 additionally failed three robustness contracts, and traversal 05 failed the repeated-parameter contract. One fixture per CWE and five repair attempts per fixture limit application diversity. Remote Semgrep rule provenance, targeted test coverage, same-host reproducibility, and lack of production performance testing further limit generalization. The five SQLi automated READY_FOR_HUMAN_REVIEW states were not automatic approvals; my September 22 bounded acceptances are separate human records and preserve the failed SQL-R01 outcomes.

Feedback requested: Is this three-fixture, category-separated supplemental evaluation sufficient for the practicum scope? Would an independent second reviewer for selected disagreements add more useful evidence than broadening to another CWE?

Methodology Paragraph Summary:

Each controlled fixture defines an attacker-controlled input, target route and vulnerable operation, synthetic data, baseline attack behavior, and benign-behavior expectations. The frozen primary protocol uses five separate initial repair calls per CWE. It holds the generator (gpt-5.2), prompt template (1.0), and model-tool access (none) constant while withholding evaluator ground truth and test implementations from the repair prompt. The same saved candidates receive syntax, Semgrep, targeted HTTP/browser security, and functional checks. SAST-only interpretation and deterministic multi-stage policy are compared on those same candidates. Primary attempts, pilot retries, the non-AI false-success control, lifecycle replay, and supplemental tests have separate denominators.

Default Semgrep CE auto did not identify the SQLi-injection target. Primary-v1 preserved that default miss and used a documented, benchmark-focused fixproof_controlled taint rule for repeatable before-and-after target comparison. SQLi ground truth remained independent of SAST and was established through the demonstrated runtime exploit.

The 15 scheduled calls are experimental attempts, not 15 independent applications or 15 unique patches. The five SQLi calls produced identical candidate source, while XSS and path traversal each produced five distinct candidate sources. This duplication is preserved as an observed model outcome and prevents the repeated SQLi code from being presented as five independent implementations.

The supplemental protocol was fixed before execution and compares each candidate with its baseline using registered security, behavioral-parity, and robustness oracles. XSS browser observations distinguish displayed text from payload execution. Inconclusive execution is not a pass. The design favors traceability and explicit decision boundaries over broad corpus realism; repeated repairs on one fixture are not independent applications. OWASP WSTG and Semgrep's run-rules documentation served as technical references for runtime-test organization and scanner configuration (OWASP Foundation, n.d.; Semgrep, n.d.).

Pearce et al. (2022) motivate treating generated code as untrusted. Pearce et al. (2023) study multi-model zero-shot repair and report functional-correctness challenges. Kulsum et al. (2024) use external compiler, sanitizer, and test feedback for iterative repair; FixProof does not claim validation feedback is new. Its bounded contribution is the auditable combination of stage-level evidence, deterministic disposition, and preserved human qualifications for three Express weaknesses. Datasets and success definitions differ, so cross-study success rates are not directly comparable.

Timeline:

This timeline records completed milestones through the September 28 Progress Report 3 submission and planned work thereafter. The Video III and peer-feedback dates reflect the Canvas schedule confirmed in September; later course dates should still be checked in live Canvas.

When	Description of Task	Status
Aug-Sep	Define problem, build three-CWE prototype and pilot, freeze primary protocol	Completed
Sep 4	Collect and verify 15 initial primary attempts	Completed
Sep 12-13	Complete original conflict reviews and post Video II	Completed
Sep 14-15	Freeze supplemental protocol, run three baselines and 15 saved candidates, record three requested follow-ups	Completed
Sep 21	Record XSS 01-02 consistency qualifications; historical checkpoint reached 116 tests	Completed
Sep 22	Record nine SQLi/traversal decisions; verify 119 tests and all 14 follow-up records	Completed

When	Description of Task	Status
Sep 23	Complete the evidence appendix and align Report 3 with the professor's benchmark and design-rationale feedback	Completed
September 24	Submit four Video II feedback responses and deploy sanitized public evidence reference	Completed
September 28	Submit Progress Report 3; official deadline October 4 at 11:59 p.m.	Completed
October 6 / 11	Post Video III on October 6; submit Peer Feedback Report III by October 11 at 11:59 p.m.	Planned
October 18 / 20 / 25	Progress Report 4, Video IV, peer feedback; deepen related work and reproducibility	Planned
November 1 / 3 / 8	Progress Report 5, Video V, peer feedback; settle final results and rehearse	Planned
November 15	Submit final presentation video, no more than 15 minutes	Planned
November 22	Submit peer feedback on final presentation	Planned
December 6	Submit final project report and reproducible package	Planned

Evaluation:

All 15 frozen primary initial attempts have complete evidence. The denominator excludes pilot retries and the constructed non-AI control. Original human conflict reviews are 10/10 complete: seven accepts under the original evidence and three requests for additional testing. Later follow-up verdicts are recorded separately.

Primary case	Initial attempts	Target SAST resolved	Security pass	Functional pass	Automated decision
Reflected XSS	5	0/5	5/5	5/5	5 need adjudication
SQL injection	5	5/5	5/5	5/5	5 ready for review
Path traversal	5	0/5	5/5	5/5	5 need adjudication
Total	15	5/15	15/15	15/15	5 ready for human review; 10 need human adjudication

Security and functional columns count candidates whose entire frozen primary suite passed; they do not count individual test cases. The supplemental-v1 results are reported separately.

Interpretation of persistent SAST findings: The target finding remained for all five XSS and all five path-traversal candidates, even though they passed the frozen primary security and functional suites. Code inspection indicates that the Semgrep rules did not appear to model the custom HTML-escaping helpers or path-containment checks as resolving the reported flow. These are SAST/runtime disagreements, not proof that the patches are fully secure or that the findings are false positives; FixProof therefore routed them to human adjudication.

Primary target-SAST resolution was 5/15 (33.3%); SAST/runtime disagreement was 10/15 (66.7%). No primary SAST false success was observed. That absence is not evidence of a general detection rate or production security. The separate pilot caught a functional regression; the constructed non-AI control exercises the false-success policy branch without entering AI-attempt metrics.

The primary schedule contains 15 model calls, but candidate-source diversity was five XSS sources, one SQLi source, and five path-traversal sources. The five SQLi attempts are retained as five calls under the protocol, while their identical code is disclosed rather than counted as five unique solutions.

The supplemental study evaluated the same 15 saved candidates against registered cases after baseline characterization. Its categories and denominators are separate from the primary study:

Category	Observations	Pass	Fail	Inconclusive	Meaning
Security	60	55	0	5	SymLink unavailable for five traversal candidates
Behavioral parity	45	40	5	0	Four XSS missing-input changes; one traversal valid-file rejection
Robustness	35	25	10	0	New SQLi and traversal input-handling contracts
Total	140	120	15	5	Mixed categories; not one security success rate

On September 22, I recorded nine human follow-up decisions that distinguish the candidates' tested security outcomes from full supplemental conformance. I accepted SQLi 01–05 for the bounded SQL-injection repair because their parameterized queries passed all three supplemental security cases and both behavioral-parity cases and returned no unauthorized rows. SQL-R01 still failed: repeated username parameters returned HTTP 200 with an empty array rather than the registered HTTP 400 response. I treated this as a disclosed, low-impact robustness limitation, not evidence that SQL injection remained exploitable. For traversal 02–05, I requested more testing because PATH-S06 was inconclusive and lexical path checks do not establish containment of symlink targets. Traversal 03 and 05 also retain their recorded robustness failures. These human judgments neither relabel failed or inconclusive cases nor overwrite any original review or primary record.

For a compact example, XSS 03 and XSS 04 both passed all three supplemental security cases, but XSS 03 passed all five behavioral-parity cases while XSS 04 passed four of five. XSS 04 changed an omitted name from the baseline visible text “Hello undefined” to “Hello “. The frozen XSS-P01 criterion supported a follow-up rejection, not a claim that the tested XSS attacks remained exploitable. The same later criterion led Tony to reject XSS 01 and 02 in separate September 21 qualifications while preserving their original acceptances.

Report Outline:

1. Problem, motivation, and research question.
2. Related work, alternatives, and specific contribution.
3. Controlled scope, threat model, and benchmark construction.
4. Architecture, model authority, validators, decision policy, and human boundary.
5. Frozen primary method, comparator, provenance, and reproducibility.
6. Primary results, separate pilot/control examples, and original human reviews.
7. Supplemental methods/results, later qualifications, limitations, and delivery feasibility.
8. Conclusion, references, evidence index, and AI-use disclosure.

References:

Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B., & Karri, R. (2022). Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions. 2022 IEEE Symposium on Security and Privacy. <https://doi.org/10.1109/SP46214.2022.9833571>

Pearce, H., Tan, B., Ahmad, B., Karri, R., & Dolan-Gavitt, B. (2023). Examining zero-shot vulnerability repair with large language models. 2023 IEEE Symposium on Security and Privacy. <https://doi.org/10.1109/SP46215.2023.10179324>

Kulsum, U., Zhu, H., Xu, B., & d'Amorim, M. (2024). A case study of LLM for automated vulnerability repair: Assessing impact of reasoning and patch validation feedback. Proceedings of the 1st ACM International Conference on AI-Powered Software. <https://doi.org/10.1145/3664646.3664770>

OWASP Foundation. (n.d.). Web Security Testing Guide, version 4.2. <https://wstg.owasp.org/v4.2/>

Semgrep. (n.d.). Run rules. <https://semgrep.dev/docs/running-rules>

Appendix:

Evidence index: docs/study-protocol-v1.md and benchmarks/primary/v1/ define the frozen primary study; data/primary_trials/v1/ and data/primary_reviews/v1/ preserve candidates and original reviews; data/evaluation/primary-report.json provides the verified primary report. docs/supplemental-protocol-v1.md, data/supplemental/v1/, and docs/supplemental-results-v1.md preserve the later protocol, observations, and 14 recorded human results. The September 22 review worksheet links the evidence used for the nine newest records. fixproof-public/ contains the sanitized static reference deployed at <https://fixproof.netlify.app/>; it is a presentation artifact, not experimental evidence. docs/README.md separates current references from historical drafts.

Verification: primary evidence, supplemental evidence, and all 14 human-record bindings were rechecked on September 28. The complete suite last passed 119 automated tests in the September 24 accuracy review. The primary report verified 15/15 attempts and 10/10 original reviews, and the supplemental report verified three baselines and 15 saved candidates. The deployed public reference separately loaded 15 primary rows, 15 supplemental rows, ten original reviews, 14 later records, filters, candidate details, the workflow image, and the PDF appendix. Four guided pilot live replays previously matched their recorded decisions. Saved-evidence verification does not repeat model calls or every historical scanner/browser experiment. A clean committed-archive rehearsal remains to be completed before final delivery.

Generative AI disclosure: OpenAI ChatGPT/Codex assisted planning, implementation, debugging, test orchestration, documentation, evidence organization, and drafting. FixProof separately used the OpenAI API to generate recorded repair candidates. Tony Tran remains responsible for verifying claims and references, reporting actual effort, making human review decisions, and approving the submitted text.